# ***Road Damage Detection: YOLO Comparison Report***

Comparative analysis of YOLOv8n, YOLOv8s and YOLO11s for detecting and localizing road damage, with hyperparameter tuning and a Streamlit deployment app.

### 1. Overview

The goal was to train and compare several YOLO models on a road-damage dataset, tune hyperparameters, and deploy the best model. Three models were trained under identical conditions (YOLOv8n, YOLOv8s, YOLO11s). Two of them (YOLO11s and YOLOv8n) were then retrained with tuned hyperparameters. All models were evaluated on the same held-out test split on mAP, precision, recall, IoU, speed, size and deployment cost.

**Headline result:** YOLO11s is the most accurate model (test mAP50-95 0.376). YOLOv8n is 3x smaller and about twice as fast on CPU, at a cost of about 2 mAP50-95 points. Hyperparameter tuning did not improve either model at the full 100-epoch schedule.

### 2. Dataset and setup

- **Source:** Roboflow Universe project road-damage-l1ju7-ftmwi (version 1, YOLOv8 format).
- **Split:** 4,930 train / 1,146 validation / 681 test images (15,024 / 3,588 / 2,004 box labels).
- **Classes (7):** alligator, block, crack, edge, longitudinal, pothole, transverse.
- **Label fix:** 35 label files contained polygons; these were converted to bounding boxes (min/max of the polygon points) so every model sees identical box labels.
- **Training (all baselines):** 100 epochs, image size 640, batch 16, AdamW, lr0 0.001, patience 30, seed 0, pretrained COCO weights, Tesla T4 GPU.

### 3. Model comparison (test split)

| Model | Precision | Recall | mAP50 | mAP50-95 |
|---|---|---|---|---|
| YOLOv8n | 0.605 | 0.565 | 0.580 | 0.357 |
| YOLOv8s | 0.613 | 0.539 | 0.590 | 0.361 |
| **YOLO11s** | **0.647** | 0.556 | **0.604** | **0.376** |
| YOLO11s (tuned) | 0.632 | 0.572 | 0.603 | 0.371 |
| YOLOv8n (tuned) | 0.602 | 0.556 | 0.576 | 0.356 |

On the validation split the order is the same: YOLO11s 0.250, YOLOv8s 0.240, YOLOv8n 0.239 mAP50-95.

**Speed, size and deployment cost**

| Model | Params (M) | GFLOPs | .pt (MB) | ONNX (MB) | GPU FPS (T4, batch 1) | GPU latency (ms) | CPU FPS (ONNX) | Train time (h) |
|---|---|---|---|---|---|---|---|---|
| YOLOv8n | 3.14 | 8.6 | 6.5 | 12.8 | 128.5 | 7.8 | 15.1 | 1.26 |
| YOLOv8s | 11.14 | 28.4 | 22.5 | 44.8 | 86.1 | 11.6 | 5.4 | 1.98 |
| YOLO11s | 9.43 | 21.4 | 19.2 | 37.9 | 89.4 | 11.2 | 7.5 | 1.83 |

YOLO11s is strictly better than YOLOv8s here: higher accuracy, fewer parameters, fewer FLOPs, smaller file, and slightly faster. There is no reason to prefer YOLOv8s.

**IoU analysis**

| Model | Ground truths matched at IoU 0.5 | Mean IoU of matched boxes | Mean best IoU per ground truth |
|---|---|---|---|
| YOLOv8n | 0.475 | 0.775 | 0.403 |
| YOLOv8s | 0.519 | 0.766 | 0.445 |
| YOLO11s | 0.504 | 0.771 | 0.433 |

When a model finds an object, its box is well placed (mean IoU about 0.77 for all three). The differences are in how many objects get found: YOLOv8s matches the most ground truths and YOLOv8n the fewest. mAP50-95 is itself an IoU-averaged metric (IoU 0.50 to 0.95), so it also reflects localization quality.

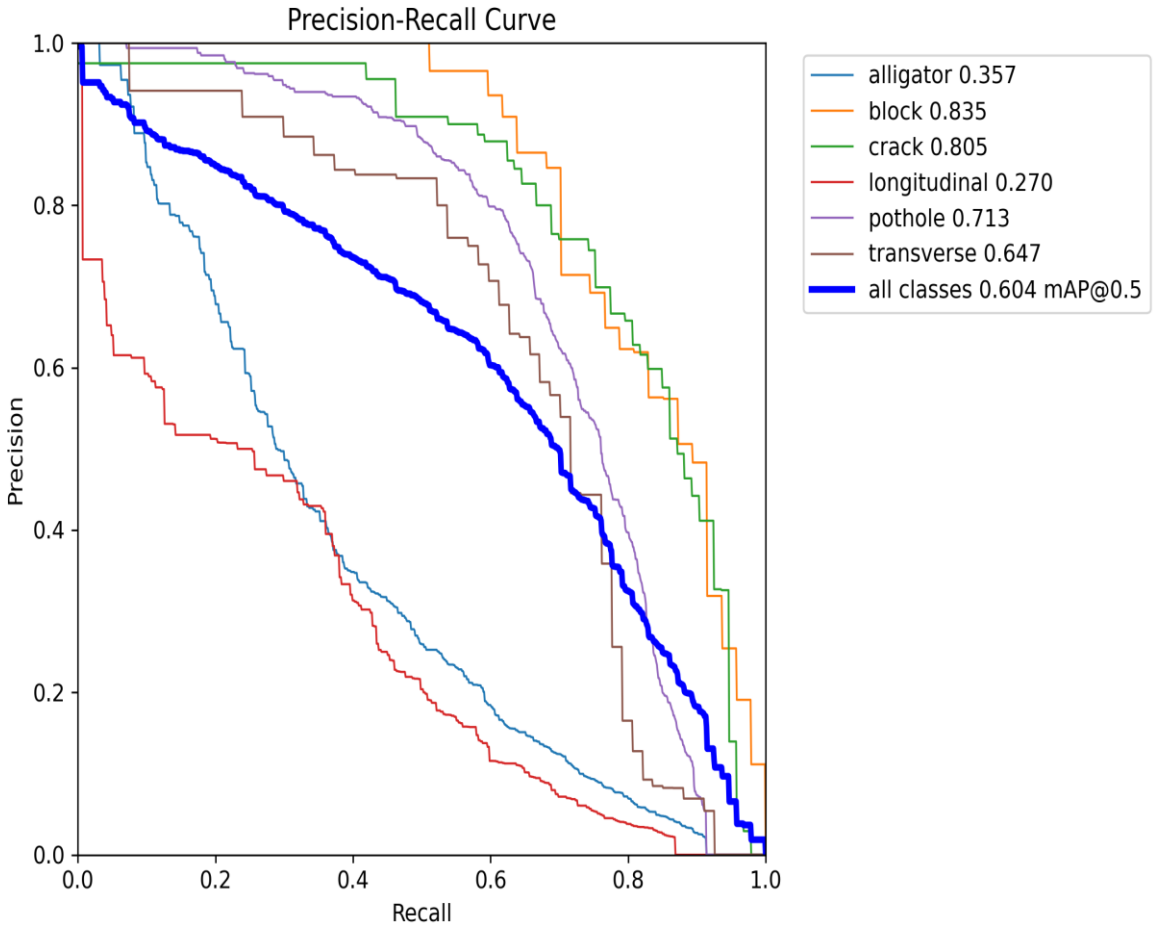
**Per-class results (test AP50-95)**

| Class | Test instances | YOLOv8n | YOLOv8s | YOLO11s |
|---|---|---|---|---|
| block | 47 | 0.598 | 0.593 | 0.642 |
| crack | 93 | 0.584 | 0.599 | 0.626 |
| pothole | 909 | 0.375 | 0.382 | 0.391 |
| transverse | 67 | 0.358 | 0.362 | 0.350 |
| alligator | 577 | 0.150 | 0.142 | 0.150 |
| longitudinal | 311 | 0.078 | 0.090 | 0.095 |

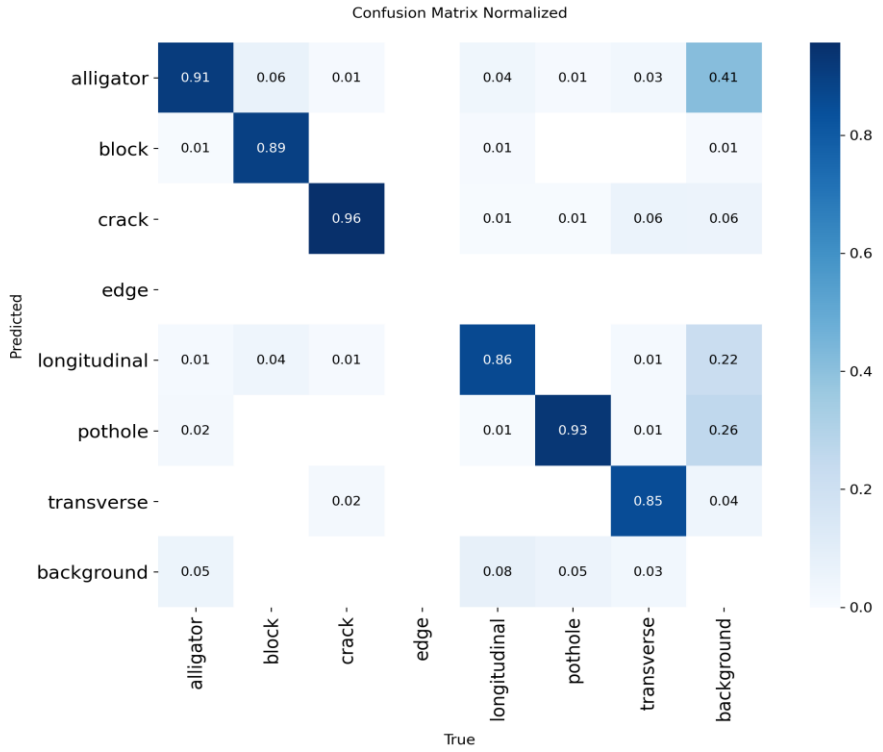
The test split has no instances of the edge class, so it is not scored there.

**Error analysis (YOLO11s, test split)**

The precision-recall curves give AP50 per class: block 0.835, crack 0.805, pothole 0.713, transverse 0.647, alligator 0.357 and longitudinal 0.270 (all classes 0.604). For alligator and longitudinal, precision falls below 0.5 before recall reaches about 0.4, which is what pulls the mean down.




The normalized confusion matrix shows few confusions between damage types (off-diagonal values of 0.01 to 0.06). Most errors involve the background: among detections with no matching label, 41% are alligator, 26% pothole and 22% longitudinal. The model therefore mostly misses objects or draws boxes where the labels have none, rather than mixing up damage types. This fits the heavily overlapping alligator, pothole and longitudinal labels in the dataset, but that explanation was not tested.




How to read the matrix: its diagonal (0.85 to 0.96) is high because Ultralytics computes it at a very low confidence threshold (the validation default is 0.001); the raw count matrices of the YOLOv8s runs show 10,000 to 22,000 background detections for alligator, longitudinal and pothole. It shows how many objects are found when every candidate box is kept, not the recall at the deployed threshold. The overall recall at the F1-optimal threshold is 0.556 (section 3).

### 4. Hyperparameter tuning

A manual grid of six trials was run on YOLOv8n with a shortened schedule (30 epochs, patience 15) and scored on the validation split:

| Trial | Change from base | Val mAP50-95 | Val mAP50 | Train time (h) |
|---|---|---|---|---|
| t2_lr_low | lr0 0.0005 | **0.2234** | 0.3834 | 0.37 |
| t5_aug_light | mosaic 0.5, scale 0.3, hsv_v 0.2 | 0.2206 | 0.3813 | 0.36 |
| t1_base | AdamW, lr0 0.001 | 0.2189 | 0.3754 | 0.37 |
| t3_sgd | SGD, lr0 0.01 | 0.2180 | 0.3792 | 0.37 |
| t6_cos_lr | cosine LR schedule | 0.2158 | 0.3762 | 0.37 |
| t4_imgsz800 | image size 800 | 0.2049 | 0.3694 | 0.55 |

The winner (AdamW, lr0 0.0005, image size 640) was then used for full 100-epoch retraining of both YOLO11s and YOLOv8n.

**Outcome:** tuning did not help at full length. YOLO11s test mAP50-95 went from 0.376 to 0.371, and YOLOv8n from 0.357 to 0.356. The 30-epoch winner beat the base by only 0.005, which is within run-to-run noise, so the selection was probably chasing noise. Two firmer conclusions from the grid: increasing the image size to 800 hurt accuracy and cost 49% more training time, and the optimizer and schedule choice matters little for this dataset.

## 5. Discussion

**Accuracy.** YOLO11s leads on mAP50, mAP50-95 and precision. The gap to the other models is small (0.015 to 0.019 mAP50-95), and every result comes from a single seed. A difference this size should be treated as suggestive, not proven; repeating training with 3 or more seeds would settle it.

**Efficiency.** YOLOv8n has 3x fewer parameters, a 6.5 MB checkpoint, 128 FPS on a T4 and 15 FPS on CPU. For a dashcam, phone or edge device without a GPU this is the more practical choice. YOLO11s still runs at about 89 FPS on a T4, which is well beyond real time, but only about 7.5 FPS on CPU.

**Hard classes.** Alligator cracking (AP50-95 about 0.15) and longitudinal cracks (about 0.09) are poorly detected by every model, with recall near 0.25 to 0.30. Likely causes, which were not tested: these damage types look similar and overlap each other, and thin or diagonal cracks have bounding boxes that contain mostly background, which makes IoU unstable after the polygon-to-box conversion. Potholes, blocks and transverse cracks are detected much better.

**Validation vs test gap.** Test scores (0.36 to 0.38) are much higher than validation scores (0.24 to 0.25). Validation contains one edge instance that every model misses, which pulls the 7-class average down, and the class mix differs between splits (for example crack AP is about 0.09 on validation vs about 0.60 on test). The two splits should not be compared directly; use the test split for model-vs-model comparison, which is what this report does.

## 6. Model selection

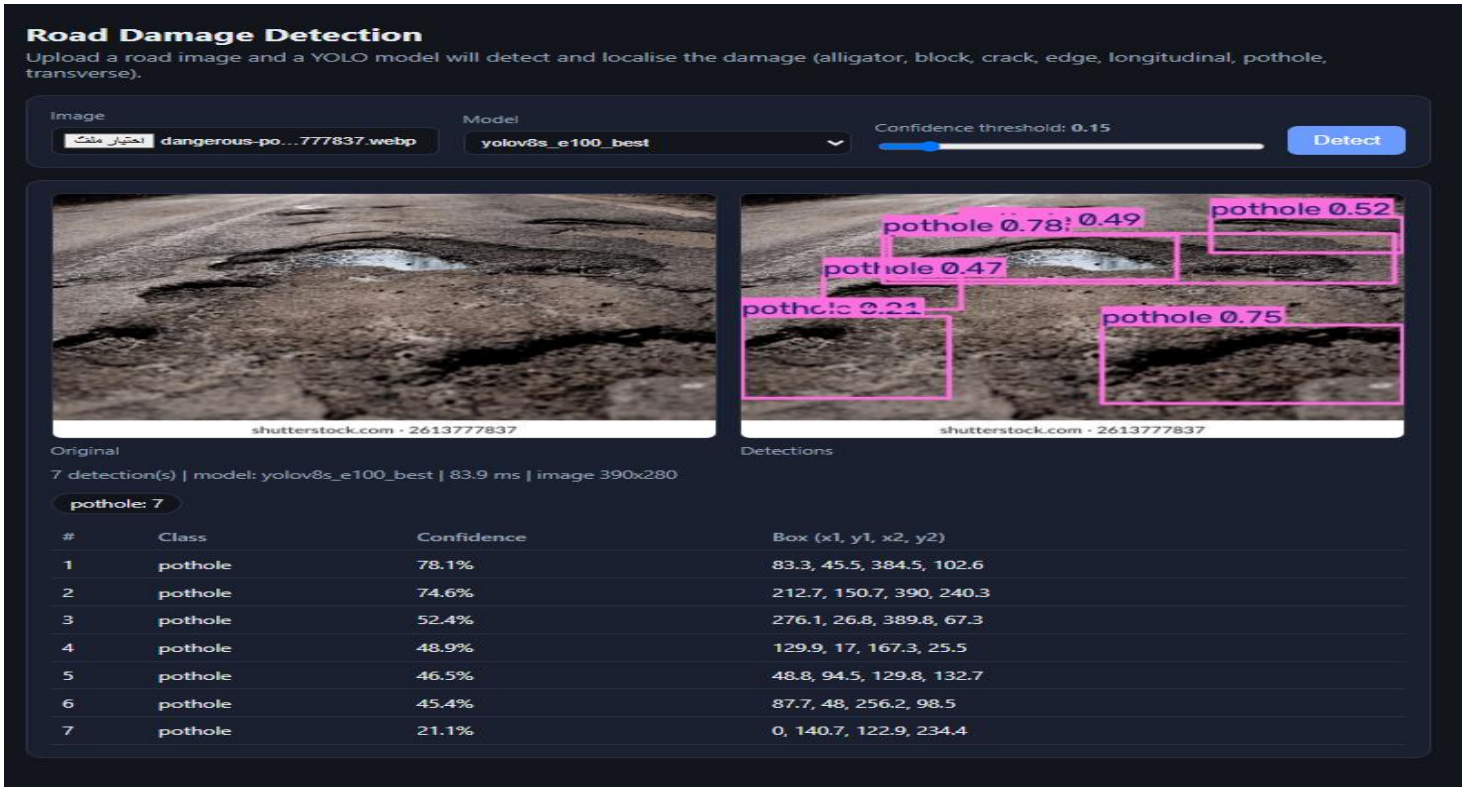
- **Deployed by default: YOLO11s (baseline, lr0 0.001).** Best accuracy of all trained models and still far faster than real time on a GPU. The tuned version is not better, so the simpler baseline is used.
- **Recommended for CPU or edge use: YOLOv8n.** About 2 mAP50-95 points lower but 3x smaller and 2x faster on CPU. The app lets you switch between weight files.
- **Not recommended: YOLOv8s.** Larger and slower than YOLO11s with lower accuracy.

## 7. Deployment app

The app is a FastAPI service (`main.py`) with a browser upload page at `/` and interactive API docs at `/docs`. It scans a `models/` folder for Ultralytics `.pt` files, loads each model once and keeps it in memory, and defaults to the YOLO11s weights. Endpoints: `GET /health` and `GET /models`

(available weights), POST /predict (JSON with class, confidence and box for each detection, counts per class, inference time and an optional base64 annotated image) and POST /predict/image (annotated JPEG only). The model and confidence threshold are chosen per request; uploads are limited to 10 MB and invalid images return clear error codes (400, 404, 413). Predictions are serialized with a lock because Ultralytics predictors are not thread-safe. A confidence of about 0.25 to 0.30 is a sensible default: the F1-optimal confidence measured in notebook 04 was 0.25 to 0.33 across the models. The service handles single images; continuous video or webcam streaming is not included.

**App screenshots.** The same stock photo of a potholed road (390x280, a stock-photo watermark is visible; the dataset also contains stock photos, so it was not checked whether this exact image is in it) run through the web page at confidence 0.15, which is deliberately low and shows many overlapping boxes. All detections are potholes: YOLOv8s finds 7 (first image), YOLOv8n finds 10 (second image) and YOLO11s finds 8 (third image). A single image is a demonstration of the app, not evidence of which model is better; the test-split metrics in section 3 are the basis for that.



## Road Damage Detection

Upload a road image and a YOLO model will detect and localise the damage (alligator, block, crack, edge, longitudinal, pothole, transverse).

Image: اختيار ملف dangerous-po…777837.webp

Model: yolov8n_e100_best

Confidence threshold: 0.15

Detect




Original




Detections

10 detection(s) | model: yolov8n_e100_best | 37.4 ms | image 390x280

pothole: 10

| # | Class | Confidence | Box (x1, y1, x2, y2) |
|---|---|---|---|
| 1 | pothole | 78.5% | 218.9, 145.5, 390, 259.8 |
| 2 | pothole | 65.1% | 274.8, 23.2, 354.3, 44.4 |
| 3 | pothole | 61.6% | 84.6, 42.7, 370.4, 98.7 |
| 4 | pothole | 55.1% | 0, 144.9, 67.5, 203.4 |
| 5 | pothole | 39.1% | 47.7, 95.4, 127.8, 130.2 |
| 6 | pothole | 25.4% | 260, 59, 362.5, 96.6 |
| 7 | pothole | 18.9% | 2.2, 46, 387.7, 259.4 |
| 8 | pothole | 16.4% | 128.2, 11.8, 169.5, 34.3 |
| 9 | pothole | 15.9% | 337.5, 36.5, 390, 64.3 |
| 10 | pothole | 15.5% | 128.2, 15.2, 164.3, 25.6 |

## Road Damage Detection

Upload a road image and a YOLO model will detect and localise the damage (alligator, block, crack, edge, longitudinal, pothole, transverse).

Image: اختيار ملف dangerous-po…777837.webp

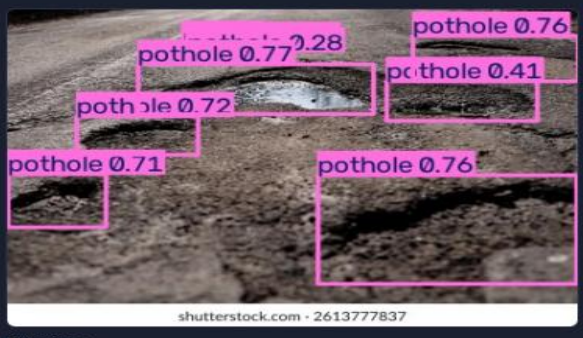
Model: yolo11s_e100_best

Confidence threshold: 0.15

Detect




Original




Detections

8 detection(s) | model: yolo11s_e100_best | 85.3 ms | image 390x280

pothole: 8

| # | Class | Confidence | Box (x1, y1, x2, y2) |
|---|---|---|---|
| 1 | pothole | 77.3% | 89.2, 48.9, 250.7, 92.1 |
| 2 | pothole | 75.6% | 277.2, 23.5, 390, 63.7 |
| 3 | pothole | 75.5% | 212.5, 146.9, 390, 242.6 |
| 4 | pothole | 71.8% | 48, 94.3, 130.4, 128.7 |
| 5 | pothole | 71.0% | 0, 145.5, 67.4, 192.3 |
| 6 | pothole | 41.0% | 259.8, 63.6, 362.5, 98.3 |
| 7 | pothole | 28.4% | 123.7, 16, 165.1, 26.2 |
| 8 | pothole | 18.6% | 120.3, 11.8, 174.8, 39.8 |

## 8. Limitations and next steps

- Single training seed per model; ranking differences under about 0.02 mAP50-95 are not statistically established.
- Only YOLOv8n was tuned directly; the best settings were transferred to YOLO11s. YOLOv8s was not tuned.
- The speed, size and IoU numbers in this report come from notebook 04, where they were entered by hand from a benchmark run whose code was not in the notebooks. A standalone `benchmark.py` now reproduces them (GPU and CPU/ONNX FPS, sizes, parameters, IoU); re-run it and replace the tables above with its output, since its IoU definitions are stated explicitly and may differ slightly from the original. GPU numbers are from a Tesla T4.
- The edge class has effectively no validation or test data and should be merged into another class or removed.
- Possible improvements: train with segmentation or oriented labels instead of boxes for thin cracks, rebalance alligator/longitudinal classes, try YOLO11m, use a longer schedule or multi-seed averaging, and export to ONNX/TensorRT for production serving.